

# Fiori Elements 開發課程 8：Interface / Projection 兩層架構

**環境：**切回 BTP ABAP Environment Trial ( 套件 **ZRAPCLOUD**，跟 fe07 的 On-Premise **S4H** 不是同一套系統——fe07 是這門課唯一的例外，這一課起恢復 fe01 ~ fe06 的環境 )

## Lecture

這一課要解決的問題

fe01 ~ fe05 用的 **ZI\_RC05\_NOTE**、fe06/fe07 查證的標準 App，全部都是單層 CDS View 直接掛 BDEF、直接曝露成 Service——這是這門課到目前為止唯一用過的模式。但這不是唯一的做法：SAP 官方對 RAP Business Object 的正式建議架構，是兩層——**Interface View** ( 跟 BDEF 綁定，技術中立，不放 UI Annotation ) + **Projection View** ( **as projection on**，搭配 Projection BDEF，這一層才放 UI Annotation、才是真正曝露給 Service 的對象 )。這一課要查證這套架構現在還是不是官方建議、什麼時候該用，並且動手把既有的 **ZI\_RC01\_TASK** ( rc01 建立至今從沒曝露成 App ) 補上 Projection 層，做出它第一個真正的 Fiori App。

查證：兩層架構沒有過時，是官方明確建議的做法

查證官方文件 ( **ABENDDIC\_PROJECTION\_VIEWS** 系列、**Business Object Interface**、**Extensibility Architecture Overview** ) 加上 openSAP 官方 RAP 課程 ( **abap-platform-rap-opensap** ) 現行教材，確認：兩層架構到現在都是官方正式建議的架構，openSAP 官方教材現在還是這樣教 ( Week 2/4 都是先建 **ZI\_RAP\_Travel\_####**——Interface，不含 Annotation，再建 **ZC\_RAP\_Travel\_####**——Projection，**as projection on**，才加 Metadata Extension )。

官方文件明講什麼時候該用兩層：「The depicted architecture is recommended if you want to make use of release contracts in your BO」——當這個 RAP BO 要當成一個穩定、可版本化、可能被其他人擴充的公開介面時 ( 像 SAP 自己交付的標準 BO 幾乎都是這樣 )，兩層分離是正式建議做法；同一個 Interface 理論上可以有多个不同的 Projection，服務不同的 App ( 例如同一個 Travel BO，一個 Projection 給內部員工完整編輯、另一個給客戶唯讀 )。官方文件也明講「如果 BO 不用這套架構，也可以直接...」——單層不是被淘汰，是「要不要那層穩定性/可重複曝露」的取捨。

對照這門課自己的 **ZI\_RC01\_TASK**：它是單層 ( DDLX 直接 annotate 在 Interface 身上 ) ——這是課程規模小、Task 沒有被多個 App 重複消費需求的刻意簡化，不是不知道有兩層這回事。

兩層架構的完整語法

**Interface View** ( **ZI\_RC01\_TASK**，rc01 已經建好，不用重做 )：BDEF 綁在這裡 ( **managed implementation in class zbp\_i\_rc01\_task unique; strict(2); ... create; update; delete; action markDone ...** )，沒有 UI Annotation。

**Projection View** ( **ZC\_RC01\_TASK**，這一課新建 )：

```
@Metadata.allowExtensions: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RC01 Task Projection View'
```

```

define root view entity ZC_RC01_TASK
  as projection on ZI_RC01_TASK
{
  key task_id,
  description,
  status,
  priority,
  due_date,
  created_at,
  created_by
}

```

**Projection BDEF ( `ZC_RC01_TASK` ) :**

```

projection;
strict ( 2 );

define behavior for ZC_RC01_TASK alias TaskProjection
{
  use create;
  use update;
  use delete;

  use action markDone;
}

```

⚠ 查證重點：**use action** 一定要顯式宣告，**Determination / Validation** 不用——官方範例明講：

「Determinations and validations are automatically triggered by the RAP application infrastructure at runtime. Therefore, there is no need to project them. But this is not the case for actions. Therefore, they always need to be explicitly exposed」。 `ZI_RC01_TASK` 的 BDEF 有 **determination setCreationInfo / validation validateStatus**，這兩個完全不用在 Projection BDEF 裡重複宣告，但 **markDone** 這個 Action 一定要用 **use action markDone;** 明講，不然畫面上不會出現這顆按鈕。

**Metadata Extension ( `ZC_RC01_TASK` )**：把原本掛在 `ZI_RC01_TASK` 上的 DDLX 內容整個搬過來 ( **annotate entity** 目標從 `ZI_RC01_TASK` 換成 `ZC_RC01_TASK` )，欄位定義一字不改。

**Service Definition ( `ZRC01_SD` ) :**

```

define service ZRC01_SD {
  expose ZC_RC01_TASK as Task;
}

```

⚠ ⚠ 核心規則：**define service** 曝露的是 **Projection ( `ZC_RC01_TASK` )**，不是 **Interface**

( `ZI_RC01_TASK` ) ——這是兩層架構存在的意義所在：Service 永遠對著 Projection 層曝露，Interface 層保持技術中立、不直接面對外部消費者，讓同一個 Interface 之後可以再長出第二個、第三個不同用途的 Projection，各自曝露不同的 Service，互不干擾。

## Eclipse ADT 建立步驟 ( 套件 ZRAPCLOUD )

1. **Projection View** : 對套件 ZRAPCLOUD 右鍵 → New → Data Definition ( 或 Core Data Services → Data Definition ) , Name 填 ZC\_RC01\_TASK , Template 選 Define View ( 這系統從 rc01 起就是 entity 語法 , 不用擔心選錯模板 ) , 貼入上面的程式碼 , 存檔→啟用 ( Ctrl+F3 )
2. **Projection BDEF** : 對 ZC\_RC01\_TASK 右鍵 → New Behavior Definition , Implementation Type 選 Projection , Root Entity 自動帶入 , 貼入程式碼 , 存檔→啟用
3. **Metadata Extension** : 對 ZC\_RC01\_TASK 右鍵 → New Metadata Extension , Name 填 ZC\_RC01\_TASK , 把 ZI\_RC01\_TASK 既有 DDLX 內容複製過來改目標名稱 , 存檔→啟用
4. **Service Definition** : 對套件右鍵 → New → Service Definition , Name 填 ZRC01\_SD , 貼入程式碼 , 存檔→啟用
5. **Service Binding** : 對 ZRC01\_SD 右鍵 → New Service Binding , Name 填 ZRC01\_SB , Binding Type 選 OData V4 - UI , 會跳出 Transport Request 選擇畫面 ( 套件不是 \$TMP , 一定要選 ) 。 ⚠️ ⚠️ 建立完是 inactive , 要先 Ctrl+F3 啟用 , Publish 按鈕才會真的生效——容易誤會「Publish」本身就是啟用 , 實際上是「建立→啟用→Publish」三個獨立步驟。

### ⚠️ 踩坑記錄

- @Metadata.allowExtensions: true 這個旗標 , 不限於「擴充別人的標準物件」才需要——fe07 學到的是擴充別人的標準 CDS View 需要這個旗標 , 這一課發現 : 任何 CDS View 只要想掛自己的 Metadata Extension , 都要有這個旗標 , 即使是自己新建的 View 也一樣 ( ZI\_RC01\_TASK\_SUMMARY 一開始漏加 , 建 Metadata Extension 時直接報錯「Annotation 'Metadata.allowExtensions' missing」) 。
- Service Binding 的「先啟用才能 Publish」這個順序容易被忽略 , 見上面步驟 5 的說明。

## 前端 : VS Code Fiori Generator Step by Step

沿用 fe01 已經連好的 TRL 系統 , 不用重新輸入 URL / 登入 :

1. Ctrl+Shift+P → 執行 Fiori: Open Application Generator
2. Template 選 List Report Page
3. Data Source 選 Connect to a System
4. System 選既有的 TRL
5. Service 搜尋框輸入 ZRC01\_SB , 選 ZRC01\_SB > ZRC01\_SD (0001) ( 這裡挑的是 Service Binding 的技術名稱 , 不是 Service Definition——同一個坑 fe01 已經記錄過 )
6. 「Download value help metadata」選 Yes
7. Main Entity 選 Task , Table Type 選 Responsive
8. Project Attributes : Module Name 填 fe08taskprojection ( ⚠️ 全小寫——Module Name 欄位不允許大寫字母 , 一開始填 fe08TaskProjection 會直接報錯「Module cannot contain capital letters」 , 跟 fe01 用的 fe01connectiontest 是同一條規則 ) , Application Title 自訂 , 其餘保留預設 ( Enable TypeScript : No ) , Project Folder Path 指到 src/ABAP\_Training\_Fiori\_Elements/ , 按 Finish
9. 產生完成後 , 開終端機 , 先 cd 進 src/ABAP\_Training\_Fiori\_Elements/fe08taskprojection/ 這個專案資料夾 ( npm start 是讀當下目錄的 package.json , 一定要先切換過去 , 不能在別的資料夾直接執行 ) , 再執行 npm start ( 背後等同 fiori run --open /test/flp.html#app-preview )
10. 終端機會依序印出 fiori-tools-proxy 連線資訊、Livereload middleware started for port ... , 最後出現 Server started / URL: http://localhost:XXXX——這才是真正就緒的訊號 , Port 不保證是 8080 : 如果 fe01 ~ fe07 留下的其他專案還在跑 ( 佔用 8080/8081... ) , 這次會自動往下一個空號遞增 , 一定要看終端機印出的實際 URL : 那一行 , 不要憑印象假設是 8080

11. 瀏覽器應該會自動開啟該 Port 的 `/test/flp.html#app-preview` ; 如果沒有跳出登入視窗 ( 代表 Reentrance Ticket 還在有效期內, 沿用之前的授權 ) , 畫面會直接顯示 List Report ; 如果卡在瀏覽器原生 Basic Auth 帳密框 ( 本課程已知的 `npm start` 認證異常, 見 fe04 講義記錄 ) , 把該 Port 的 process 砍掉重開即可排除

同時跑多個 `npm start` 沒有問題——每個專案各自的 Port 不同, 可以讓 fe08 / fe09 兩個開發伺服器同時開著, 方便直接切換分頁對照兩邊畫面的差異

⚠️ ⚠️ 實測結果跟原本預期不同, 這是這一課最有價值的發現: List Report 表格上有 `Mark Done` 這個 Action 按鈕 ( 來自 `use action markDone;` ) , 但沒有 **Create / Edit** 按鈕——原本以為 Projection BDEF 有 `use create;` `use update;` `use delete;` 就會產生對應按鈕, 實測推翻了這個假設。

原因: 呼應 RAP Cloud 課程 ( rc05 ) 學過的關鍵結論——「**Draft 是 Fiori Elements Create/Edit 按鈕的前提**」。ZI\_RC01\_TASK 的 BDEF 只有 `create;` `update;` `delete;` , 沒有宣告 `with draft;`——即使 BDEF 技術上支援這些操作 ( EML / ADT 單元測試都能正常呼叫 ) , Fiori Elements 範本的標準 UI 不會因此產生 Create/Edit 按鈕; Draft 不是「錦上添花」的功能, 是範本判斷「這個 Entity 能不能在畫面上編輯」的前提條件。反觀 `markDone` 是 Action ( 不是 create/update 這類基本操作 ) , 不受這條規則限制, 正常顯示。

教訓: 規劃這一課時只想著「兩層架構語法對不對」, 沒有意識到「BDEF 有沒有 Draft」這個更根本的前提會直接決定畫面上有沒有基本 CRUD 按鈕——這是兩件獨立的事, 兩層架構本身跟 Draft 完全無關,

ZC\_RC01\_TASK 這個 Projection 如果想要有 Create/Edit 按鈕, 要回頭幫 ZI\_RC01\_TASK 的 BDEF 加上 `with draft;` ( 這已經超出這一課「兩層架構」本身的範圍, 留給讀者當延伸練習 ) 。

## 學習目標

- 知道 Interface / Projection 兩層架構是官方現行建議, 不是過時做法; 能講出「什麼時候該用兩層」的判斷準則 ( Release Contract / 同一個 BO 需要被多個 App 用不同方式曝露 )
- 能寫出 Projection View ( `as projection on` ) 與 Projection BDEF ( `projection;` + `use` 關鍵字 ) 的語法
- 知道 Determination / Validation 不用在 Projection BDEF 裡重複宣告, 但 Action 一定要用 `use action` 顯式曝露
- 知道 `define service` 要曝露 Projection 層, 不是 Interface 層——這是兩層架構真正發揮作用的地方
- 知道任何要掛 Metadata Extension 的 CDS View 都要有 `@Metadata.allowExtensions: true` , 不限於擴充標準物件的情境
- 知道 Service Binding 建立後要先啟用 ( `Ctrl+F3` ) 才能正常 Publish

## 物件清單

套件 ZRAPCLOUD ( BTP Trial, 非 \$TMP, 需要傳輸請求 ) :

| 物件           | 型別                       | 說明                                                                                                                                    |
|--------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| ZI_RC01_TASK | DDLS + BDEF              | 既有 Interface View ( rc01 建立, 這一課沒有修改 )                                                                                                |
| ZC_RC01_TASK | DDLS ( Projection View ) | <code>as projection on ZI_RC01_TASK</code>                                                                                            |
| ZC_RC01_TASK | BDEF ( Projection )      | <code>projection;</code> <code>use create;</code> <code>use update;</code> <code>use delete;</code> <code>use action markDone;</code> |

| 物件           | 型別                              | 說明                                 |
|--------------|---------------------------------|------------------------------------|
| ZC_RC01_TASK | DDLX ( Metadata Extension )     | UI Annotation · 從 ZI_RC01_TASK 搬過來 |
| ZRC01_SD     | Service Definition              | expose ZC_RC01_TASK as Task;       |
| ZRC01_SB     | Service Binding ( OData V4-UI ) | 已 Publish                          |

## 動手練習

輪到你了：

- 想一想：如果要再加一個「唯讀、給主管看的」Projection ( 例如 ZC\_RC01\_TASK\_DISPLAY · 只有 use create 拿掉、use update / use delete 也拿掉，只保留讀取 )，語法上怎麼寫？這樣兩個 Projection 會不會互相干擾？
- 試著在 Projection View 裡拿掉某個欄位 ( 例如 created\_by )，看 Metadata Extension 對這個已經不存在的欄位下 Annotation 會發生什麼事 ( 提示：這是驗證「Projection 可以是 Interface 的子集」這個特性 )
- 對照 fe07 學過的「標準 App 的 CDS View Extension」——如果 ZI\_RC01\_TASK 之後也想開放給別人擴充，需要補上 fe07 教過的哪個 Annotation？

## 驗證方式

後端物件已在 Eclipse 完整建立、啟用、Service Binding 已 Publish 成功 ( 截圖為證 )。前端 Fiori Elements App 的產生與畫面驗證見下一節動手操作 ( 沿用 fe01 的 VS Code Generator 流程，Service 選 ZRC01\_SB > ZRC01\_SD → Task )。

## 思考題

- ZI\_RC01\_TASK 原本掛的那份 DDLX 沒有刪除，技術上變成「多餘」( 沒有 Service 直接曝露 Interface，這份 Annotation 沒有實際作用 )。你覺得應不應該刪掉它？留著有沒有風險？
- 這一課的 Projection BDEF 用 use create; use update; use delete;，全部原封不動繼承 Interface 的行為。如果想讓 Projection 層「限縮」某個欄位的可編輯性 ( 例如讓 priority 在這個 Projection 唯讀，但 Interface 層仍然可編輯 )，語法上要怎麼做？( 提示：查一下 Projection BDEF 裡 field(readonly) 的用法 )

## 答案

見 ZC\_RC01\_TASK ( DDLS / BDEF / DDLX )、ZRC01\_SD、ZRC01\_SB，皆建立於 BTP Trial 系統套件 ZRAPCLOUD。ZI\_RC01\_TASK 沿用既有物件未修改。